

Activity 3

Log Analysis

Collaborators. Claude (for L^AT_EX styling and grammar checking)

Overview

Suppose you are on the security/incident management team of a (supposed) company named buttercup. You have access to a set of **system logs** (secure.log) and **web logs** (access.log) from your company infrastructure consisting of:

- 1 mail server named mailsv
- 3 web servers named www1, www2, www3

It is your job to identify successful/failed attempts to hack, scan, login, or access protected information on the servers.

The logs are at the google drive link I posted on Discord. Reading/parsing the logs manually will be PAINFUL. You could write code to do this, but that will also take some time and effort. Instead, we will use a popular tool, Splunk, to help us with our job.



Figure 1: Magic Quadrant for Security Information and Event Management

Getting Started

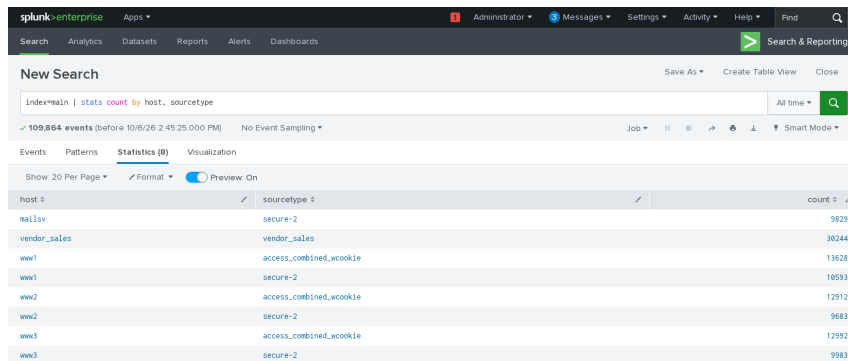
1. You will use Splunk, a popular log analysis tool, that you can download from: <http://www.splunk.com/> → Choose Free Splunk → Software Download → Download Splunk Enterprise. You will need to provide your information to Splunk before you download. Download and install Splunk Enterprise on your computer and start running it (Start Splunk Enterprise and launch Splunk Web).
2. In your browser, go to your newly installed Splunk Web on your computer at: <http://localhost:8000>.
3. Import the data downloaded above (the entire zip file) into Splunk following these steps (Add Data, Use Segment in Path to identify Host).
WARNING: do this only once or you will have multiple copies of the data in your analysis, and your count will be incorrect. Look at how to load data in the example in this link first... <http://docs.splunk.com/Documentation/Splunk/latest/SearchTutorial/GetthetutorialdataintoSplunk>
 After the import, if you cannot see the data, look at the time frame of your analysis and select as far back in time as possible. The default is only 24 hours.
You must use only the data set I provided. Using other data sets will have different results when grading.

Understand the Data

Use Splunk's "Search" feature to explore the data.

<http://docs.splunk.com/Documentation/Splunk/latest/SearchTutorial/Aboutthesearchapp>

My setup. I installed Splunk Enterprise 9.4.1 and imported tutorialdata.zip once, taking the host name from the folder in the path (mailsv, www1, www2, www3, vendor_sales). All searches below use the time range **All time**. The event counts match the number of lines in each log file, so the data was imported exactly once. Splunk parses access.log automatically (sourcetype access_combined_wcookie, with fields such as clientip, uri_path, status, useragent), but for secure.log (sourcetype secure-2) I extract user and src_ip myself with rex.



host	sourcetype	count
mailsv	secure-2	9829
vendor_sales	vendor_sales	30244
www1	access_combined_wcookie	13628
www1	secure-2	10593
www2	access_combined_wcookie	12912
www2	secure-2	9683
www3	access_combined_wcookie	12992
www3	secure-2	9983

Figure 2: Imported events per host and sourcetype

Part I. Can you find people trying to break into the servers?

Use Splunk's "Search" feature to try to answer the questions below.

Hint 1: On linux servers, secure.log contains security-related information. Typically in response to incidents, it is one of the first files people look at to see if there are compromises. Read this to see what to look for on Linux (this secure.log file is available in google drive).

<https://zeltser.com/security-incident-log-review-checklist/>

Hint 2: To process the logs for analysis, first parse it using regular expressions to "extract fields" and turn unstructured data into structured data.

Answer the following questions and provide evidence with your answer.

Problem 1. How many hackers are trying to get access to our servers? And how many attempts are there? Explain/define how you count distinct hackers.

Solution

Answer: 182 hackers made 33,069 failed login attempts.

How I count a hacker. Every failed SSH login in secure.log looks like Failed password for [invalid user] <user> from <ip> port <port> ssh2. I extract the source IP with rex and treat **one distinct external source IP = one hacker**. The three internal IPs (10.x.x.x) are not counted because they belong to our own staff: they log in successfully hundreds of times and only fail occasionally (typos), see Figure 4. No external IP ever has an Accepted password line, so all 33,069 attempts failed.

```
index=main sourcetype=secure-2 "failed password"
| rex "for (invalid user )?(?<user>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| search NOT src_ip="10.*"
| stats count AS attempts dc(src_ip) AS distinct_hackers
```

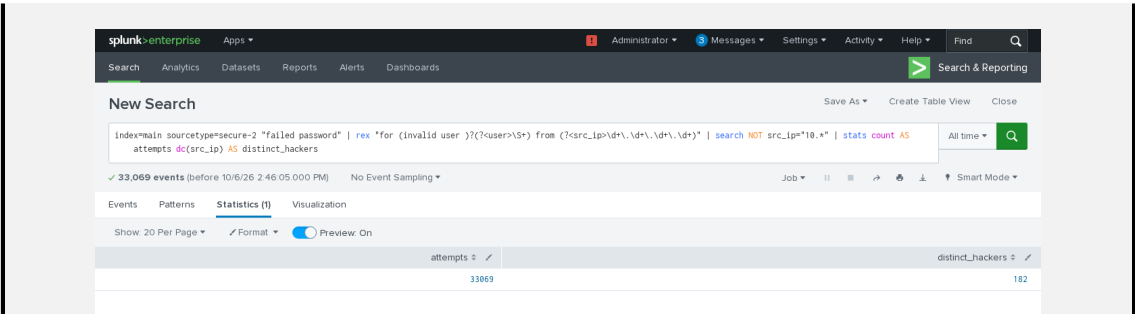


Figure 3: Number of attempts and distinct hacker IPs

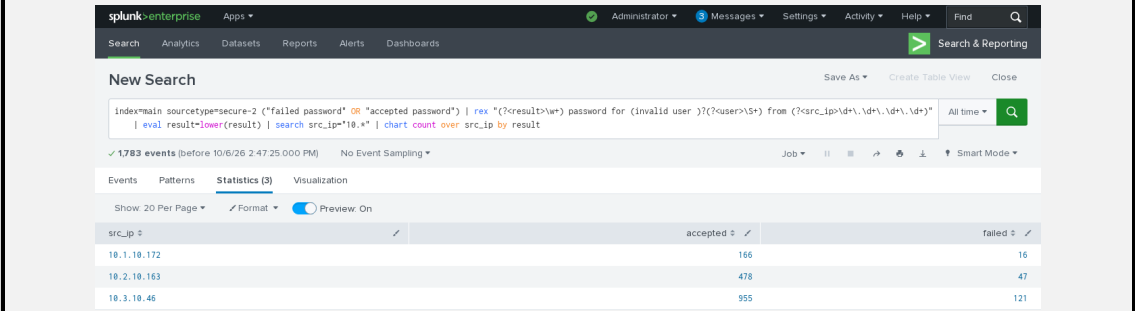


Figure 4: Internal IPs: mostly successful logins (employees), so excluded

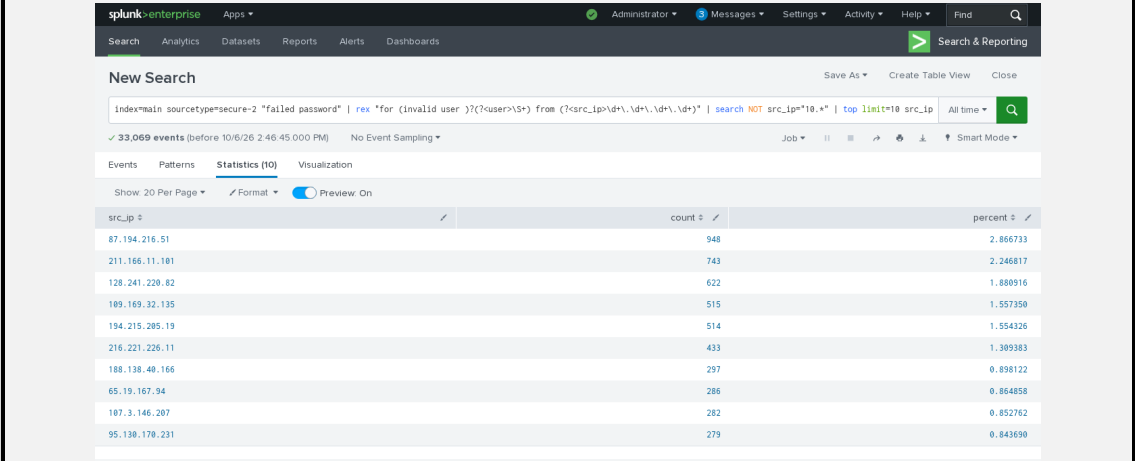


Figure 5: Top 10 hacker IPs (| top limit=10 src_ip)

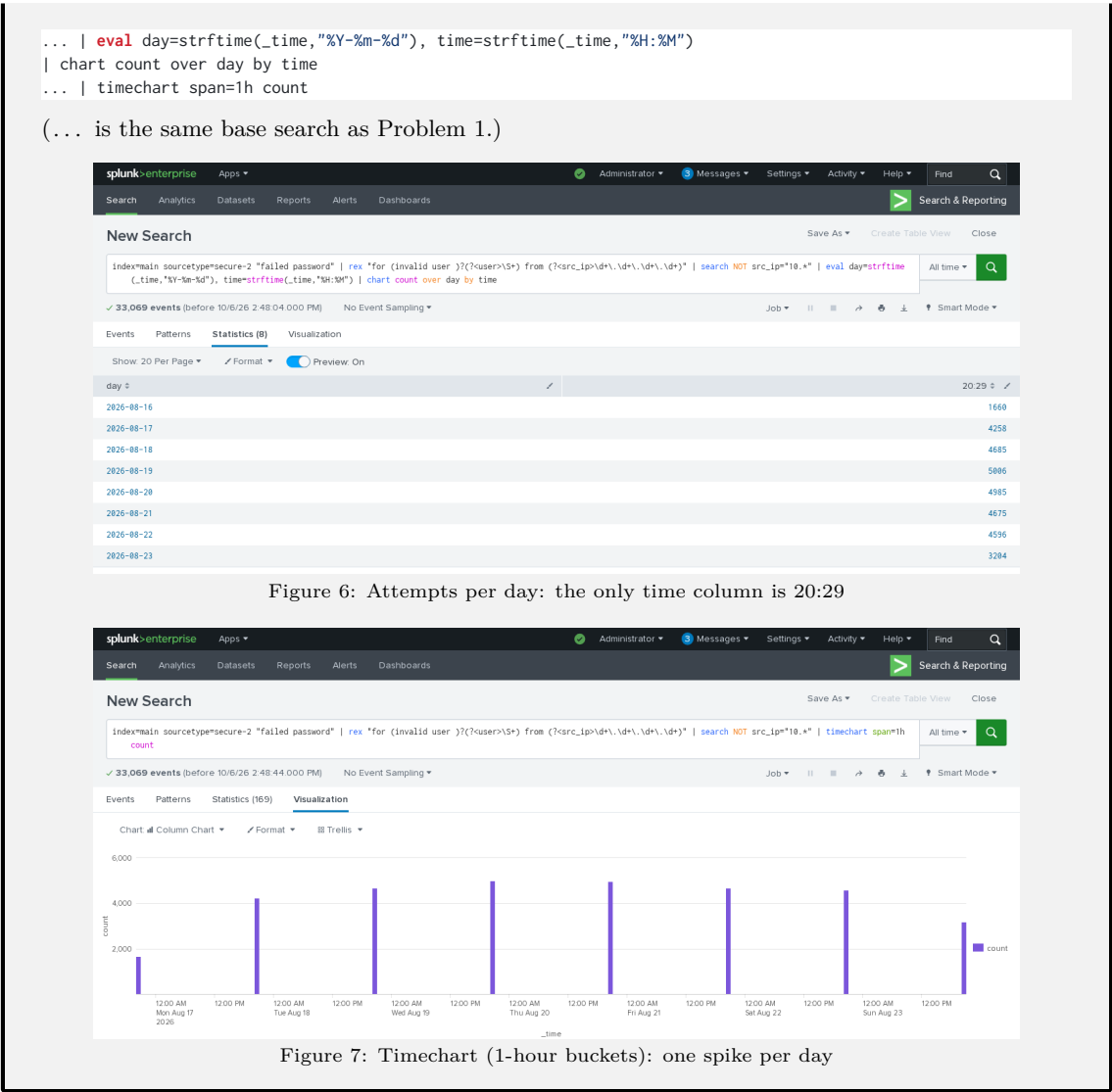
Each hacker IP tried between 63 and 948 times (about 182 on average), which is clearly an automated brute-force/dictionary attack, not a user mistyping a password. **Limitation:** counting by IP is only an estimate. One attacker can use many IPs (botnet/proxies), and one IP can be shared by many people (NAT).

Problem 2. What time do hackers appear to try to hack our servers?

Solution

Answer: every night at about 20:29 (8:29 PM), from 16 to 23 Aug 2026.

All 33,069 attempts fall between 20:29:35 and 20:29:38 of each day, so the attack is a short burst that happens at the same time every day. This looks like a scheduled script (e.g. a cron job) that runs in the evening when the admins are probably not watching. The busiest day is 19 Aug (5,006 attempts); the first and last days are smaller because the log starts/ends in the middle of the period.



Problem 3. Which server (mailsv, www1, www2, www3) had the most attempts?

Solution

Answer: **www1** with 8,746 attempts (26.4%).

The attack is spread almost evenly across all servers, so the hackers are scanning everything:

Host	Attempts
www1	8,746
www3	8,220
mailsv	8,111
www2	7,992

```
... | stats count by host | sort -count
```

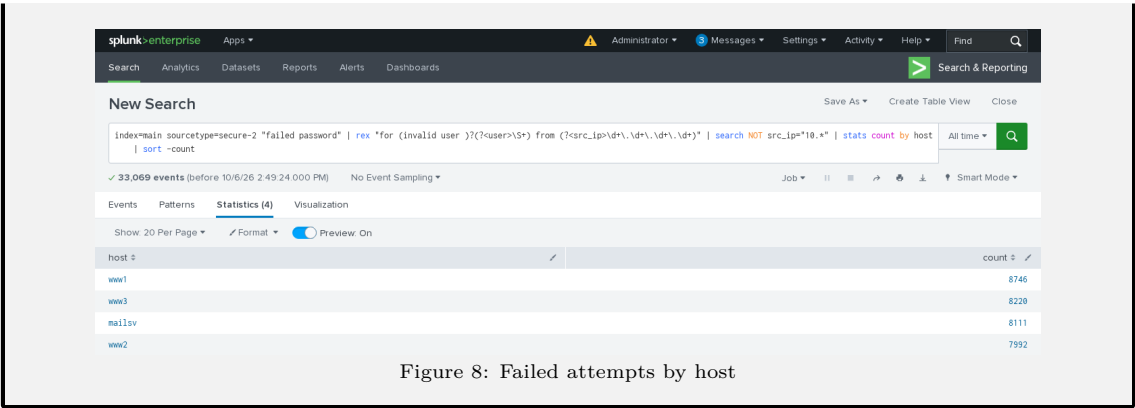


Figure 8: Failed attempts by host

Problem 4. What is the most popular account that hackers use to try to break in?

Solution

Answer: root with 1,493 attempts, followed by administrator (1,020) and admin (938).

These are the default/privileged account names that exist on almost every system, so they are the first guesses in a dictionary attack. In total the hackers tried 151 different usernames, and 24,011 of the attempts (73%) were for invalid users, i.e. accounts that do not even exist on our servers.

... | top limit=10 user

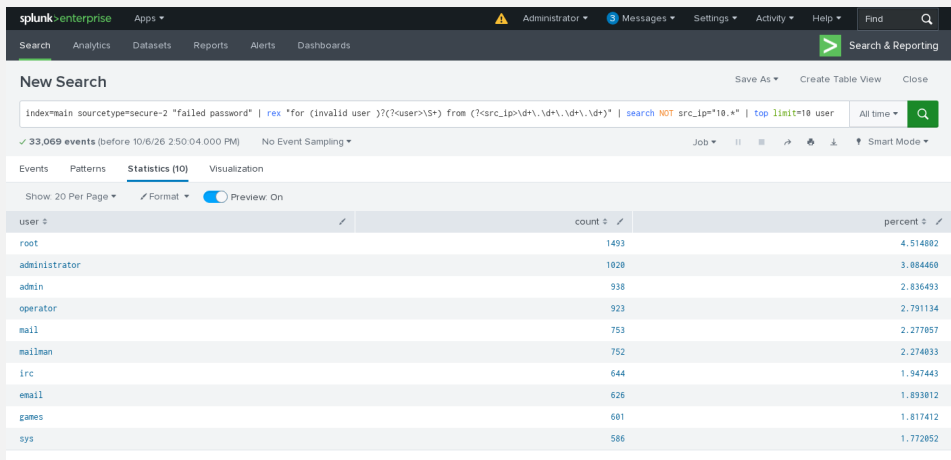


Figure 9: Top 10 usernames used by hackers

Part II. Sensitive Files on Web Servers

Hint: On web servers, access.log contains web access-related information. Typically in response to incidents, it is one of the first files people look at to see if there are compromises. Read this to see what to look for on Web Servers.

<https://zeltser.com/security-incident-log-review-checklist/>

Problem 5. Can you find attempts to get access to sensitive information from our web servers? How many attempts were there?

Solution

Answer: yes, 212 attempts (from 118 different IPs) to download sensitive files, and **none of them succeeded** (all returned 404 Not Found).

Normal customers only use the shop pages (/product.screen, /category.screen, /cart.do, /oldlink, /cart/*.do). Filtering those out leaves 610 requests to pages that do not exist on the site, and every one of them got 404. This is someone probing the web servers for hidden files. The ones that are clearly after sensitive data are /passwords.pdf, /hidden/anna_nicole.html, and /rush/signals.zip:

```
index=main sourcetype=access_combined_wcookie
NOT uri_path IN ("/product.screen", "/cart.do", "/category.screen",
                "/oldlink", "/cart/success.do", "/cart/error.do")
| stats count by uri_path status

index=main sourcetype=access_combined_wcookie
uri_path IN ("/passwords.pdf", "/hidden/anna_nicole.html", "/rush/signals.zip")
| stats count dc(clientip) AS distinct_ips values(status) AS status by uri_path

\myspace
```

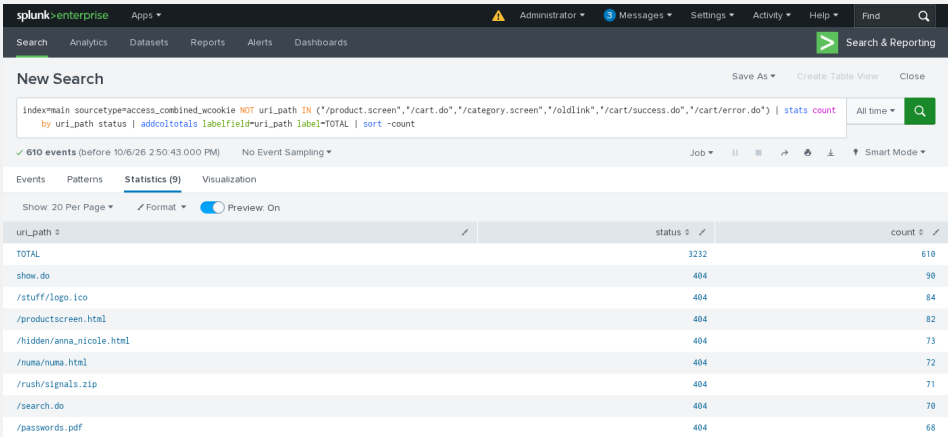


Figure 10: All requests outside the normal shop pages (all 404)

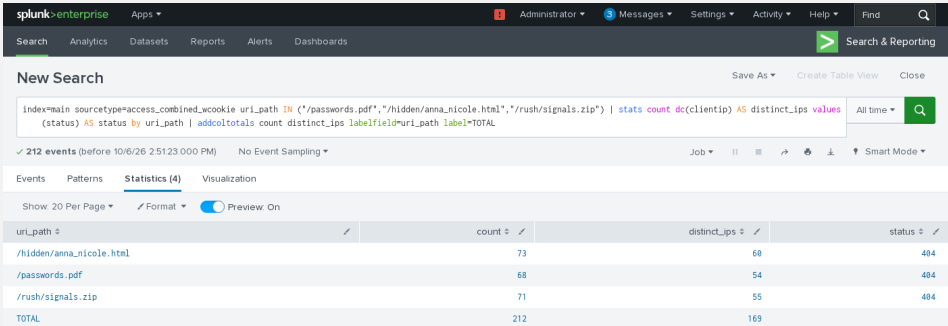


Figure 11: Requests for sensitive files

Problem 6. What resource/file are hackers looking for?

Solution

The main target is `/passwords.pdf` (68 requests), a file that would contain passwords. They also look for a hidden page `/hidden/anna_nicole.html` (73) and an archive `/rush/signals.zip` (71), which could contain internal/confidential data. The other 404 paths (`show.do`, `search.do`, `/productscreen.html`, `/numa/numa.html`, `/stuff/logo.ico`) look like guessing common page names to discover what exists on the server.

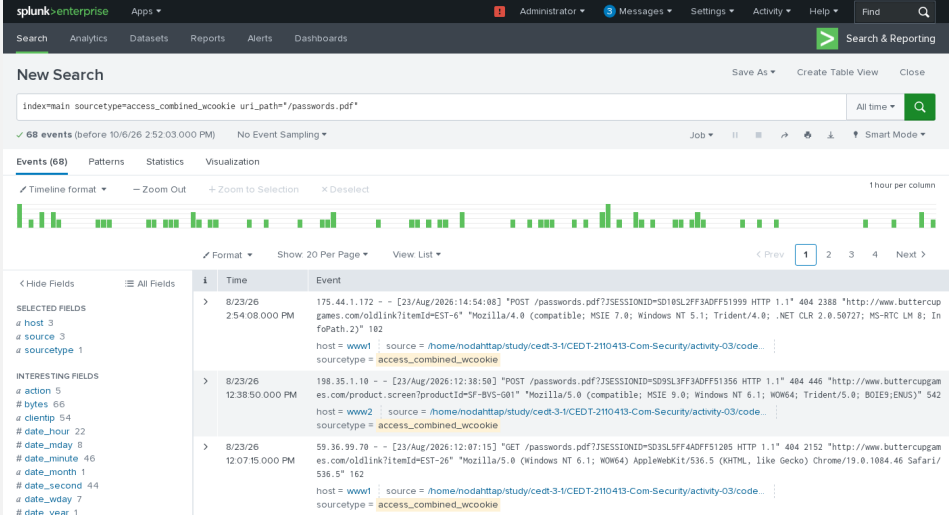


Figure 12: Raw events requesting `/passwords.pdf` (status 404)

Part III. Are there bots crawling our websites?

Problem 7. Can you find any bots crawling our websites?

Solution

Answer: yes, 2,346 requests (about 5.9% of all web traffic) come from 3 bots, found by searching the User-Agent for bot/Netcraft:

Bot	Requests	Distinct IPs	What it is
Googlebot (3 UA strings)	1,466	119	Google search crawler
NetcraftSurveyAgent	497	53	Netcraft web survey
YandexBot	383	50	Yandex search crawler

```
index=main sourcetype=access_combined_wcookie
(useragent="*bot*" OR useragent="*Netcraft*")
| eval bot=case(like(useragent,"%Googlebot%"),"Googlebot",
               like(useragent,"%YandexBot%"),"YandexBot",
               like(useragent,"%Netcraft%"),"Netcraft")
| stats count dc(clientip) AS distinct_ips by bot, useragent
```

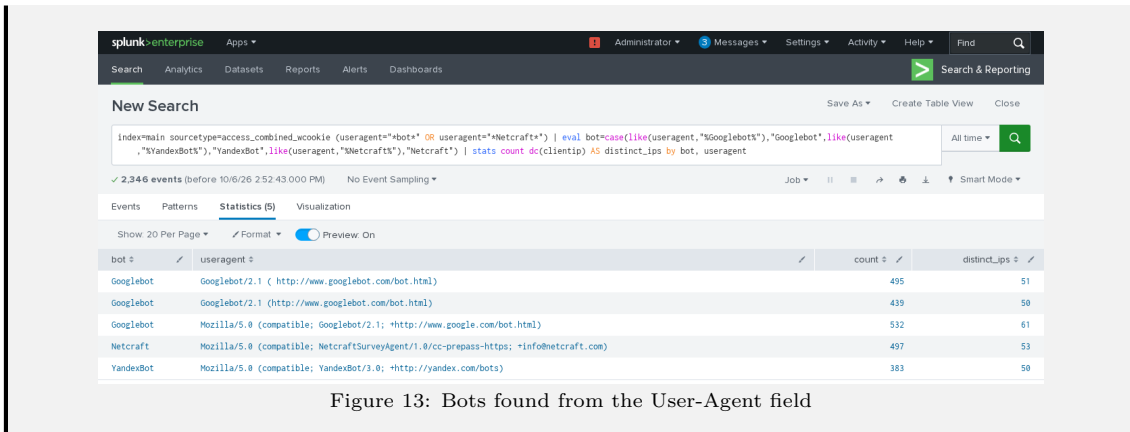


Figure 13: Bots found from the User-Agent field

Problem 8. What are they doing on the site? (*Hint:* Look for User-Agent in the web access.logs.)

Solution

Mostly crawling the shop, but some of the behaviour is not what a real search-engine bot does.

- **Crawling/indexing (normal):** most requests are GET on product, category and old-link pages (/product.screen 461, /oldlink 354, /category.screen 315), i.e. reading the catalogue to index it.
- **Using the shopping cart (suspicious):** they also POST to /cart.do and /cart/success.do. By the action field they addtocart 359 times and purchase 338 times. A crawler should never fill forms or buy things.
- **Probing sensitive files:** 12 bot requests went to /rush/signals.zip, /hidden/anna_nicole.html and /passwords.pdf.
- **Fake identity:** none of the 1,466 “Googlebot” requests come from Google’s crawler range 66.249.64.0/19; they come from 119 random IPs. The User-Agent header is just text that the client chooses, so these are most likely scrapers/attackers **pretending to be bots**.

```
... | chart count over action by bot
... | eval page=method." ".uri_path | chart count over page by bot
index=main sourcetype=access_combined_wcookie useragent="*Googlebot*"
| eval in_google_range=if(cidrmatch("66.249.64.0/19",clientip),"yes","no")
| stats count by in_google_range
```

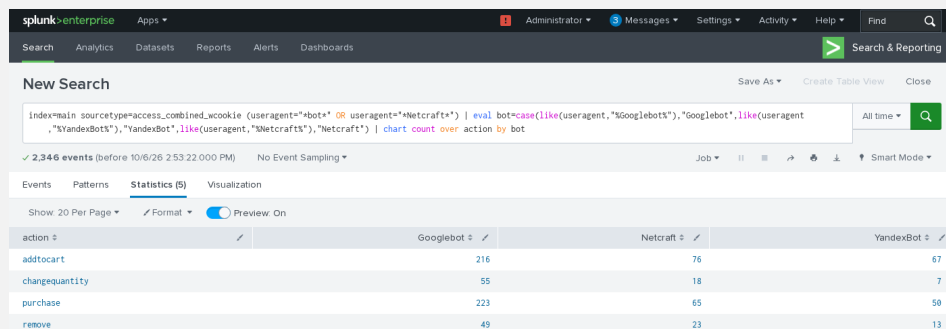
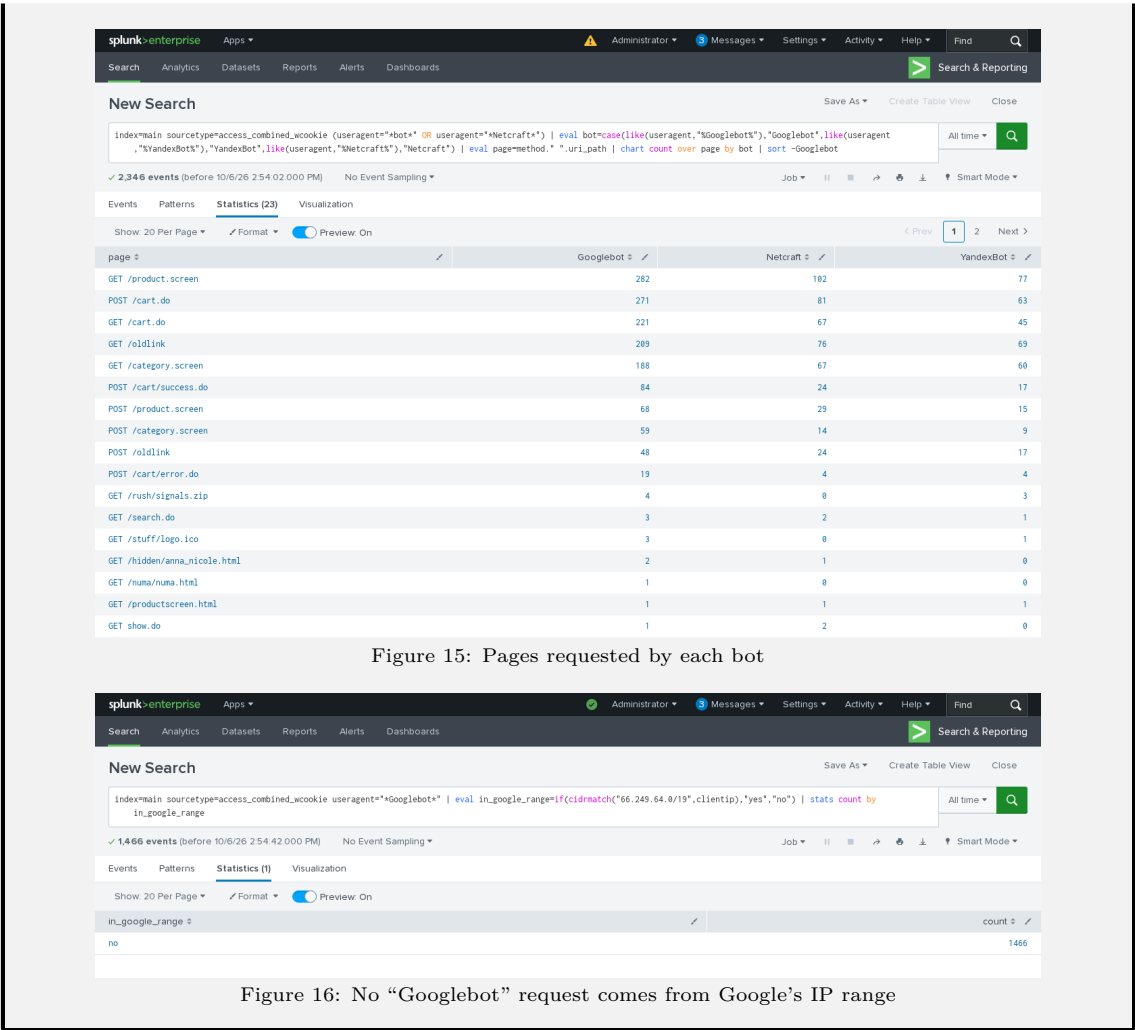


Figure 14: Shop actions done by the bots



Cleaning up

Shut down splunk on your notebook (otherwise, it will keep running in the background). Mac users should follow Unix instructions. The splunk command is located at /Applications/Splunk/bin/. You may uninstall splunk after you are done with this assignment.
<http://docs.splunk.com/Documentation/Splunk/latest/Admin/StartSplunk>

Submission

Upload your answers and evidence for your answers (i.e., screenshots, commands) as a pdf.